

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное учреждение высшего образования
«Дальневосточный федеральный университет»
(ДВФУ)

ИНСТИТУТ МАТЕМАТИКИ И КОМПЬЮТЕРНЫХ ТЕХНОЛОГИЙ

Департамент математики и компьютерного моделирования

Отчет
к лабораторной работе №3 по дисциплине «Суперкомпьютеры и параллельное программирование»

По теме: «Ожидание завершения асинхронного метода с использованием тайм-аута»

Вариант 20

Выполнил: студент группы Б9122-01.03.02мкт

_____ Чирич И.С.
подпись *Ф.И.О.*

Проверил: Доцент

_____ Селезнев Т.Э.
подпись *Ф.И.О*

г. Владивосток
2025

Содержание

Постановка задачи **3**

Индивидуальное задание **4**
Написание программного кода 4
Тестирование 5

Вопросы для защиты работы **7**

Выводы **8**

Постановка задачи

Цель данной лабораторной работы: научиться использовать механизм ожидания завершения работы асинхронного метода с использованием типа `IAsyncResult` и тайм-аута.

Задачи лабораторной работы:

- Научиться использовать механизм тайм-аутов
- Научиться выводить информацию о ходе выполнения асинхронного метода
- Научиться отслеживать выполнение асинхронного метода

Индивидуальное задание

Написание программного кода

Воспользуемся кодом лабораторной работы №2 и изменим условие остановки тела цикла. В предыдущем коде цикл пользовательского интерфейса был с условием на проверку поля `IsCompleted` асинхронного интерфейса `IAsyncResult`:

```
for (int i = 0; !ar.IsCompleted ; i++)
{
        Console.Write('\b');
        Console.Write(SYMBOLS[i % 4]);
        Thread.Sleep(100);
}
```

Изменим эту логику, сделав бесконечный цикл, в теле которого используется дескриптор ожидания. Его можно получить через метод `AsyncWaitHandle.WaitOne` типа `IAsyncResult`. Первым аргументом он принимает количество времени, которое дескриптор будет ожидать результат. По истечении этого времени он вернет `false` и выполнение продолжит вызывающий поток:

```
for (int i = 0; ; i++)
{
        Console.Write('\b');
        Console.Write(SYMBOLS[i % 4]);
        if(ar.AsyncWaitHandle.WaitOne(100, false))
        {
                Console.WriteLine("\Вычисленияn выполнены.");
                break;
        }
}
```

Итоговый код в листинге ниже:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading;
using System.Threading.Tasks;

namespace async_delegates
{
        internal class Program
        {
                static void Main(string[] args)
                {
                        string[] SYMBOLS = { "-", "\\", "|", "/" };
                        int SIZE = 1_000_000;
                        Func<int, List<int>> dl = DoHardWork;
                        IAsyncResult ar = dl.BeginInvoke(SIZE, null, null);
                        Console.WriteLine("Вычисления запущены...");
```

```
                        for (int i = 0; ; i++)
                        {
                                Console.Write('\b');
                                Console.Write(SYMBOLS[i % 4]);
                                if(ar.AsyncWaitHandle.WaitOne(100, false))
                                {
                                        Console.WriteLine("\Вычислениян выполнены.");
                                        break;
                                }
                        }
                        List<int> result = dl.EndInvoke(ar);
                        Console.WriteLine("Длина полученного массива: {0}", result.
                            Count());
                        Console.WriteLine("Длина входного массива: {0}", SIZE);
                        Console.ReadKey();
                }
                //
                static List<int> DoHardWork(int size)
                {
                        int MAX_VALUE = 20_000_000;
                        Random random = new Random();
                        List<int> list = new List<int>();
                        for (int i = 0; i < size; i++)
                        {
                                list.Add(random.Next(MAX_VALUE));
                        }
                        List<int> newList = new List<int>();
                        for (int i = 0; i < size; i++)
                        {
                                if (list[i] % 6 == 0)
                                {
                                        newList.Add(list[i]);
                                }
                        }
                // делаем' остановку' на' '3000ms, чтобы' симулировать' сложную''
                    работу
                        Thread.Sleep(3000);
                        return newList;
                }
        }
}
```

Тестирование

Запуск программы осуществим командой, которая скомпилирует проект и запустит его в среде выполнения:

```
dotnet build; dotnet run
```

Вывод программы:

```
Вычисления запущены...
-
Вычисления выполнены.
Длина полученного массива: 166685
Длина входного массива: 1000000
```

Формат вывода не изменился. При одном и том же значении ссида псевдорандомной функции результат не меняется, что подтверждает работоспособность программного кода.

Вопросы для защиты работы

1. **Для чего применяется тип `IAsyncresult`?**

   `IAsyncresult` применяется для реализации асинхронной работы с методами, выполняемыми в отдельных потоках. Без него не было бы возможности получать состояние результата выполняемого потока, что потенциально приводит к блокировке вызывающего кода.

2. **Как реализовать ожидание завершения выполнения асинхронного метода с использованием тайм-аута?**

   - Вызвать `BeginInvoke` для запуска кода в отдельном потоке
   - Получить дескриптор ожидания через `IAsyncResult.AsyncWaitHandler`
   - Через метод `WaitOne` задать время ожидания результата метода

3. **Поясните назначение метода `WaitOne()`**

   Метод `WaitOne()` - один из методов дескриптора ожидания. Он позволяет указать время ожидания результата выполняемого кода и продолжить главный поток при его отсутствии.

Выводы

В данной лабораторной работе изучен один из способов работы с асинхронными методами при помощи `IAsyncResult` и дескриптора ожидания, ассоциированного с ним - `AsyncWaitHandle`.